Taller: Divide y Conquista y Teorema Maestro

Objetivo:
Diseñar algoritmos de divide y conquista (dividir, conquistar y combinar), plantear su recurrencia de costo y resolverla con el teorema maestro.
Parte 1 (Problemas 1 a 3): diseño de algoritmos. Parte 2: aplicación del teorema maestro.

Problema 1: Conteo de Inversiones

Contexto y propósito:
Contar inversiones mide qué tan desordenado está un arreglo: es el número de intercambios entre vecinos que haría falta para ordenarlo. Se usa, por ejemplo, para comparar dos rankings: si dos personas ordenan las mismas películas según su preferencia, el número de inversiones entre sus listas indica cuánto difieren sus gustos. La fuerza bruta revisa todos los pares en $\Theta(n^2)$. Lo que se pretende es que descubra cómo Merge Sort puede contar las inversiones «de paso», mientras mezcla. La idea clave es la del significado insuficiente (guía, ejercicio 3): si la función devuelve solo un número, combinar las dos mitades sigue costando $\Theta(n^2)$; hay que enriquecer lo que devuelve cada llamada.

Enunciado:
Dado un arreglo a[0..N−1] de N enteros, una inversión es un par de índices (i, j) con i < j y a[i] > a[j]. Determine el número total de inversiones del arreglo.

Entrada:
- Un entero N
- Una lista de N enteros

Salida:
- Número de inversiones

Ejemplo:
Entrada:
5
2 4 1 3 5
Salida:
3

1. Describa las tres fases del algoritmo: ¿cómo divide el problema?, ¿qué resuelve cada llamada recursiva (postcondición de la función)? y ¿cómo combina los resultados?

2. ¿Por qué no basta con que la función devuelva solo el número de inversiones? ¿Qué más debe devolver cada llamada recursiva para poder combinar eficientemente?

3. Explique los casos base:

4. Escriba la recurrencia de costo T(n); identifique a, b y f(n), y resuélvala con el teorema maestro indicando el caso:

5. Implemente la solución en Python (leyendo la entrada estándar con el formato de Entrada):

Problema 2: Índice Fijo

Contexto y propósito:
Este problema es una búsqueda binaria disfrazada: no se busca un valor dado, sino una posición que cumple una propiedad. Lo que se pretende es que identifique qué propiedad del arreglo permite, mirando solo el elemento del centro, descartar una mitad completa sin revisarla, y que justifique por qué la respuesta no puede estar en la mitad descartada. Sin esa justificación el algoritmo no es correcto, aunque funcione en el ejemplo. Conecta con el ejercicio 1 de la guía (búsqueda binaria, a = 1).

Enunciado:
Dado un arreglo a[0..N−1] de N enteros distintos ordenados de forma estrictamente creciente (pueden ser negativos), encuentre un índice i (contando desde 0) tal que a[i] = i. Si hay varios, basta con imprimir uno de ellos; si no existe, imprima "NO EXISTE". La solución debe tener complejidad O(log N).

Entrada:
- Un entero N
- Una lista de N enteros distintos en orden creciente

Salida:
- Un índice i tal que a[i] = i o "NO EXISTE"

Ejemplo:
Entrada:
7
-10 -5 0 3 7 9 12
Salida:
3

1. Describa las tres fases del algoritmo: ¿cómo divide el problema?, ¿qué resuelve cada llamada recursiva (postcondición de la función)? y ¿cómo combina los resultados?

2. Explique los casos base:

3. Escriba la recurrencia de costo T(n); identifique a, b y f(n), y resuélvala con el teorema maestro indicando el caso:

4. Implemente la solución en Python (leyendo la entrada estándar con el formato de Entrada):

Problema 3: Elemento Mayoritario

Contexto y propósito:
Decidir si hay una mayoría aparece en sistemas de votación y en tolerancia a fallas: varias réplicas de un servidor responden la misma consulta y se acepta la respuesta de la mayoría. La restricción de comparar solo por igualdad modela datos que no tienen un orden natural (imágenes, documentos, objetos) e impide ordenar el arreglo. Lo que se pretende es que descubra qué puede decir, y qué no, el mayoritario de cada mitad sobre el mayoritario del arreglo completo, de modo que combinar consista en verificar pocos candidatos. Compare al final su recurrencia de costo con la de Merge Sort.

Enunciado:
Dado un arreglo de N elementos, determine si existe un elemento mayoritario, es decir, un elemento que aparezca más de N/2 veces, e imprímalo; si no existe, imprima "NO HAY MAYORITARIO".
Restricción: los elementos solo se pueden comparar por igualdad (a = b); no se pueden ordenar ni comparar con < o >.

Entrada:
- Un entero N
- Una lista de N elementos

Salida:
- El elemento mayoritario o "NO HAY MAYORITARIO"

Ejemplo:
Entrada:
7
3 1 3 3 2 3 3
Salida:
3

1. Describa las tres fases del algoritmo: ¿cómo divide el problema?, ¿qué resuelve cada llamada recursiva (postcondición de la función)? y ¿cómo combina los resultados?

2. Explique los casos base:

3. Escriba la recurrencia de costo T(n); identifique a, b y f(n), y resuélvala con el teorema maestro indicando el caso:

4. Implemente la solución en Python (leyendo la entrada estándar con el formato de Entrada):

Parte 2: Teorema Maestro

Contexto y propósito:
Lo que se pretende es que aplique el teorema maestro de forma sistemática: identificar a, b y f(n), calcular $n^{\log_b a}$ y compararlo con f(n) antes de concluir. Las recurrencias están elegidas para compararse entre sí: las tres primeras solo cambian f(n) y recorren los tres casos; las siguientes cambian a y b; y las últimas exigen la versión general del teorema o no lo admiten, para que reconozca sus límites (guía, sección 7).

Instrucciones:
Recuerde la forma $T(n) = a \cdot T(n/b) + f(n)$, con $a \geq 1$ y $b > 1$ constantes. Para cada recurrencia identifique a, b, f(n) y $n^{\log_b a}$; indique qué caso del teorema maestro aplica o por qué no aplica, y dé el orden de T(n).

Recurrencia	a	b	f(n)	$n^{\log_b a}$	Caso	T(n)
$T(n) = 4T(n/2) + n$						
$T(n) = 4T(n/2) + n^2$						
$T(n) = 4T(n/2) + n^3$						
$T(n) = T(n/2) + 1$						
$T(n) = 3T(n/3) + n$						
$T(n) = 9T(n/3) + n$						
$T(n) = 2T(n/4) + \sqrt{n}$						
$T(n) = 2T(n/2) + n \log n$						
$T(n) = T(n - 1) + n$						
$T(n) = 2T(n/2) + n / \log n$						

1. Para las recurrencias en las que el teorema maestro no aplica, explique por qué no aplica y cómo obtendría su orden (por ejemplo, con el árbol de recursión):

2. Compare $T(n) = 2T(n/2) + \Theta(1)$ y $T(n) = T(n/2) + \Theta(1)$ como costos de dos versiones recursivas para calcular b^n reduciendo el exponente a la mitad. Resuélvalas con el teorema maestro. ¿Qué error de código produce la primera?

3. ¿Por qué un algoritmo de divide y conquista casi nunca se beneficia de memoización?